

GEHC Edison Interview Prep

Bhatiya Kavya Vishnukumar · Round I desk reference

Edison Intern

GE HealthCare

Readable layout

Labels used in this document. **Confirmed** = official email or multiple candidate reports. **Repeated** = 2+ independent GFG/repo reports. **One-off** = a single write-up — useful, not a promise. **Practice** = written for rehearsal; nobody reported this question. **Inference** = reasoned from the interviewer's profile. Nothing here is a leaked question. The question bank is built from GEHC campus write-ups already in this repo plus your own resume surface area.

Slot logistics

Date & time	Friday, 18 September 2026, 10:20 IST Confirmed
Platform	Microsoft Teams — meeting organised by Ramana Kumar Reddy A V , Talent Acquisition, GEHC South Asia Confirmed
Round	Round I, panel: Gajanan More (Senior Software Architect, GEHC Bengaluru) Confirmed
Expected length	~15–20 minutes. Slots run on a 20-minute grid (10:00, 10:20, 10:40...), so the panel cannot overrun. Inference from slot grid
Role	Edison Intern Role Confirmed
Your identifiers	Reg. U24AI038 · kavyabhatiya44@gmail.com

Teams setup — do this by 10:05.

- Join from a **laptop**, wired or strong Wi-Fi. Phone hotspot charged as backup.
- Camera **on**, plain background, light in front of you not behind you. Test audio in Teams' device settings.
- Display name = **Bhatiya Kavya Vishnukumar** (matches their records; "Kavya" alone may not).
- Join at **10:17**. If nobody admits you by 10:25, wait — the prior slot may have run long. Email Ramana only after 10:35.
- Have open, minimised: your resume PDF, and one browser tab with the LogiFlow repo/deployment. Close everything else, silence notifications.
- Keep a pen and blank A4 sheet — if he asks you to explain a design, you can sketch and hold it up or narrate structurally.

Process context

- Round I here is a **single-panel technical conversation with one senior architect**, not a coding test. **Inference from panel seniority**
- Across SVNIT GEHC accounts, the technical round is **panel-dependent**: some panels drill projects and logic without live coding, others ask you to code or design with hints. **Confirmed — senior SVNIT 2025**
- GFG SVNIT Aug 2024 (EEDP tech intern) reported a ~30-minute technical: **C++ OOP** (copy constructor, virtual functions, overriding), then projects, then a light ML question ("ChatGPT uses which type of ML?"). **Confirmed**

— GFG

- Another GFG on-campus account: deep project questions **plus** OOP **plus** a Two-Sum-style problem and a list-mutation coding question. **Repeated**
- EEDP FTE write-ups: implement **runtime polymorphism live**, OSI layers, DB locks and threads, project depth. **Confirmed for FTE, heavier than intern rounds**
- HR is usually a separate later round, but Round I may include a light why GE HealthCare. Be ready; keep it to 40 seconds. **Repeated**
- **Edison, one line:** GE HealthCare's AI and digital intelligence platform — AI and analytics over imaging and device data so clinicians get faster, better-supported decisions. **Confirmed positioning**

In a 20-minute slot, the biggest risk is not difficulty — it is over-talking. Every answer 60–90 seconds, then stop. Silence after your answer is his thinking time, not your cue to keep going. If you fill it, you lose the questions where you would have scored.

Highest-probability question bank

3.1 Opening and fit

- Tell me about yourself. **Confirmed — every account** → model answer §4.1
- Why GE HealthCare? Why Edison? **Repeated** → model answer §4.2
- You are in AI branch, second year — why software/Edison and not pure research? **Practice**
- Are you comfortable with Bengaluru? **Repeated** → “Yes, no constraints.”
- What do you know about GE HealthCare's products? **Repeated** → imaging (MRI, CT, ultrasound, X-ray), patient monitoring, and Edison as the AI/analytics layer across them.

3.2 LogiFlow — expect 6–10 minutes here

- Walk me through LogiFlow. **Confirmed pattern** → model answer §4.3
- Draw / describe the architecture. What talks to what? **Inference — architect panel** → Next.js → FastAPI → multi-objective ranking + HistGB delay-risk → PostgreSQL; Gemini assist/explain only; Docker packages the service (do not volunteer Cloud Run — not on current resume).
- What did *you* personally build versus your teammates? **Repeated — ownership probe** → You founded LogiFlow: road pipeline, hybrid scoring, HistGB delay service, planner/reports, Dockerfile. Name components precisely. Never claim every mode alone.
- 73% accuracy is not high. Why should anyone trust this model? **Inference — likely, prepare it** → model answer §4.4
- Why HistGradientBoosting / CatBoost and not a neural network? **Practice** → tabular data; boosting dominates; resume lists CatBoost; delay service is HistGB with stratified holdout + 5-fold.
- How do you optimise across four objectives at once — delivery time, reliability, cost, sustainability? **Inference — his TOGAF/trade-off instinct** → normalise each objective, weight per user constraint, rank a Pareto-ish candidate set — not one fake global optimum.
- Where does Gemini actually sit? Is it making the decision? **Practice — and an important honesty test** → No. Optimiser + classifier decide; Gemini explains/assists. If Gemini fails, ranking still works.
- Why PostgreSQL and not MongoDB? **Repeated — both on resume** → LogiFlow needs joins (routes/legs/hubs/costs); Mongo fits document-shaped data elsewhere.

- Why FastAPI and not Django here, when you used Django in MindSettler? **Practice — strong differentiator** → model answer §4.6
- Docker — what is actually in the image? **Confirmed on resume Tools** → model answer §4.5
- What broke? What would you rebuild? **Repeated** → class imbalance made accuracy look fine while missing delays — moved to recall/ROC-AUC as the real metrics.
- How did you test it? **Repeated — reliability probe** → be honest about coverage; stratified holdout + 5-fold for the delay model; say what you did not automate.

3.3 AirHelp, MindSettler, SMSRF

- AirHelp — what does it do and why Gemma instead of a hosted API? **Practice** → multilingual airport passenger assistance on FastAPI + Next.js; Gemma is open-weight so it can be self-hosted, cheaper and controllable — relevant when data cannot leave the environment.
- MindSettler — how does role-based access work? **Repeated — auth probe** → Django REST with roles (counsellor / client / admin), permissions checked per endpoint, session scheduling constrained by role; SQL schema for sessions and availability.
- Your research — explain NSGA-II simply, and what “multi-stakeholder” means. **Practice** → genetic algorithm that keeps a non-dominated front (trade-offs, not one weighted answer). Multi-stakeholder = competing objectives across parties. MovieLens 100K (+ trends on 1M): up to **+12.6% novelty, +20.9% serenity vs NSGA-3**.
- Multi-objective optimisation appears in both LogiFlow and your research — is that deliberate? **Inference** → Yes, and say so: it is your through-line. Real systems have competing objectives; picking a defensible trade-off is the actual engineering.

3.4 OOP and CS fundamentals

- Copy constructor — when is it called? Shallow vs deep copy? **Confirmed — GFG SVNIT Aug 2024**
- Virtual functions — what problem do they solve? How does the vtable work? **Confirmed — GFG**
- Difference between overloading and overriding. **Confirmed — GFG**
- Implement runtime polymorphism — write it. **Confirmed for EEDP FTE, possible here** → drill in §5
- What is the *purpose* of OOP, not the four pillars? **Repeated across repo mocks** → “Keep data and the behaviour that owns it together so a change stays local. Reuse is a side effect, not the goal.”
- Abstract class vs interface. Why a virtual destructor? **Practice**
- DBMS: normalisation, indexing, ACID, what a transaction guarantees. **Repeated**
- OS: process vs thread; race condition; deadlock conditions. **Repeated**
- CN: OSI layers; TCP vs UDP; what happens when you hit a URL. **Repeated — deep OSI in FTE rounds**
- What type of ML does ChatGPT use? **One-off but literally asked at SVNIT** → “Self-supervised pretraining of a transformer on next-token prediction, then supervised fine-tuning and RLHF for alignment.”

3.5 Behavioural (short-slot versions)

- Biggest challenge in your project and how you handled it. **Repeated**
- Tell me about a disagreement in your team. **Repeated** → resolve with evidence/benchmark, not seniority.
- Strength and weakness, with reasons. **Confirmed — GFG HR** → weakness must be real plus the correction you are running.

- How do you learn something new fast? **Practice** → point at Docker packaging / HistGB + CatBoost iteration during Solution Challenge timelines.

Model answers (60–90 seconds each)

4.1 Intro

YOU SAY

YOU SAY

Good morning sir. I am Kavya Bhatiya, B.Tech in Artificial Intelligence at SVNIT Surat, CGPA 9.05 through fourth semester. My strongest project is **LogiFlow**, a multimodal AI logistics platform — it optimises road, rail, air and sea routes on delivery time, reliability, cost and sustainability instead of only the fastest path. I built FastAPI backend pipelines with PostgreSQL, a HistGradientBoosting delay-risk service — **73 percent accuracy, 68 percent delayed-shipment recall, 78.3 percent ROC-AUC** — and Next.js with Gemini on the product surface. It made the **global Top 106 in the Google Solution Challenge 2026**. I also work on SMSRF, a multi-stakeholder recommender using NSGA-II, and I keep fundamentals sharp with DSA — **470-plus** LeetCode problems, best contest rating 1620. What I enjoy most is turning a model into a system someone can actually rely on.

4.2 Why GE HealthCare / why Edison

YOU SAY

YOU SAY

Two reasons, one technical and one personal. Technically, Edison is GE HealthCare's AI and analytics platform over imaging and device data — and that is the hardest version of the problem I have been practising. In LogiFlow, if my delay prediction is wrong, someone loses money. On Edison, a clinician is making a decision with it, so the same model needs a much higher bar: it has to be reliable, testable, and explainable to the person acting on it. That constraint is what I want to learn to build under, and I cannot learn it from a hackathon. Personally — GE HealthCare's software is inside machines that touch millions of patients, and I would rather spend my first internship on code where correctness genuinely matters than on something disposable.

4.3 LogiFlow product pitch

YOU SAY

YOU SAY

The problem: when a shipment moves between cities, a planner chooses among road, rail, air and sea — options differ on cost, transit time, delay risk, and sustainability. Today that is mostly habit and phone calls. LogiFlow takes origin, destination, cargo and constraints, and returns a **ranked set of routes with trade-offs visible**, plus predicted delay risk.

Architecture: Next.js frontend, FastAPI backend, PostgreSQL for route and operational data. Inside the backend: multi-objective ranking on delivery time, reliability, cost and sustainability; and a HistGradientBoostingClassifier for delay-risk. Gemini assists on the surface — it does not decide the ranking. Docker packages the service. Global Top 106, Google Solution Challenge 2026.

4.4 Defending the delay model's metrics

YOU SAY

YOU SAY

Honest numbers first: HistGradientBoostingClassifier, **73.0% accuracy, 68.0% recall on delayed shipments, 78.3% ROC-AUC**, with stratified holdout and 5-fold validation. Accuracy is the metric I care about least here, because delays are the minority class — a model that always predicts “on time” looks fine on accuracy and is useless. The metric that matters is **recall on delays**. 68% means we catch roughly two of three real delays; 78.3% ROC-AUC says ranking beats chance across thresholds.

I use it as a **risk signal inside ranking**, not a binary verdict. To improve: more history, better weather/congestion features, calibrated probabilities, thresholds driven by the cost of a missed delay versus a false alarm.

4.5 Docker (resume claim) — hosting only if asked

YOU SAY

YOU SAY

The backend is a Docker image: Python base, locked dependencies, FastAPI app, model artefact copied in, uvicorn on the configured port. Secrets stay out of the image — runtime env, gitignored dotenv locally. Docker buys the same artefact locally and in production and keeps the filesystem honest. If asked about production hosting: the team deploys that containerised API on Cloud Run; I will not invent concurrency or min-instance numbers I did not set myself.

4.6 FastAPI vs Django — why different stacks

YOU SAY

YOU SAY

I have used both for different shapes. **MindSettler** is counselling session management — CRUD, roles, scheduling, auth — so Django REST fit: ORM, auth and permissions come built in. **LogiFlow** is computation-heavy with outbound Gemini calls — FastAPI gave async handling, Pydantic validation at the boundary, and OpenAPI for the frontend. Trade-off: with FastAPI I wrote more of my own auth/data access; worth it because the heavy work is optimisation and inference.

OOP and DSA drills for a short slot

5.1 Runtime polymorphism — write this from memory

Confirmed — asked live in EEDP rounds

```
class Shape {
public:
    virtual double area() const = 0;        // pure virtual
    virtual ~Shape() = default;            // virtual dtor: safe delete via base ptr
};
```

```
class Circle : public Shape {
    double r;
public:
    explicit Circle(double r) : r(r) {}
    double area() const override { return 3.14159 * r * r; }
};
```

```
class Rect : public Shape {
    double w, h;
public:
    Rect(double w, double h) : w(w), h(h) {}
    double area() const override { return w * h; }
};
```

```
// dispatch resolved at run time through the vtable
void print(const Shape& s) { std::cout << s.area() << "\n"; }
```

Say while writing: “`virtual` makes the call resolve on the *dynamic* type, through a vtable pointer stored in the object, so `print` works for any future shape without changing `print`. The virtual destructor is not optional — without it, deleting a `Circle` through a `Shape*` is undefined behaviour and leaks the derived part.”

5.2 Two Sum, OOP-wrapped

Confirmed — GEHC EID 2024 candidates were told to use classes, not bare functions

```
class Solution {
public:
    std::vector<int> twoSum(const std::vector<int>& nums, int target) {
        std::unordered_map<int, int> seen; // value -> index
        for (int i = 0; i < (int)nums.size(); ++i) {
            int need = target - nums[i];
            auto it = seen.find(need);
            if (it != seen.end()) return {it->second, i};
            seen[nums[i]] = i;
        }
        return {}; // no pair found
    }
};
```

Narrate: brute force is $O(n^2)$; one pass with a hash map is **$O(n)$ time, $O(n)$ space**. Then dry-run `[2, 7, 11, 15]`, target `9` out loud. Handle the “no answer” case explicitly — interviewers watch for that.

5.3 Rapid-fire answers to have ready

- **Copy constructor:** called when an object is initialised from another object of the same type (also on pass/return by value). Default is a member-wise *shallow* copy — if the class owns a raw pointer you must write a *deep* copy or you get a double free.
- **Overloading vs overriding:** overloading = same name, different parameters, resolved at *compile* time. Overriding = derived class replaces a `virtual` base method, resolved at *run* time.
- **Abstract class vs interface:** abstract class can hold state and partial implementation; an interface (all-pure-virtual class in C++) is only a contract.

- **Process vs thread:** separate address space vs shared address space within one process — which is exactly why threads need synchronisation and processes need IPC.
- **Race condition:** two threads interleaving on shared mutable state without synchronisation; fix with a mutex or by removing the shared mutability.
- **ACID:** atomicity, consistency, isolation, durability — the guarantees a transaction gives you.
- **Index:** a secondary structure (usually a B-tree) that makes reads on a column fast at the cost of slower writes and extra storage.
- **Normalisation:** remove redundancy so an update happens in one place; denormalise deliberately when read performance demands it.
- **TCP vs UDP:** ordered, reliable, connection-oriented vs fire-and-forget with lower latency.

The healthcare / reliability angle

Line 1 — the stakes shift. “In logistics, a wrong prediction costs money and you retry. In healthcare, a clinician acted on it — so the same model needs a different bar: conservative defaults, an audit trail, and an explanation the user can check.”

Line 2 — why explainability was already in my project. “I put a plain-language explanation on every LogiFlow ranking because a planner will not trust a number they cannot interrogate. That is the same reason clinical AI has to explain itself — it is not a nice-to-have, it is what makes the output usable.”

Line 3 — reliability as a habit. “The thing I would carry into a healthcare codebase is the assumption that the model *will* be wrong sometimes, so the system has to behave sensibly when it is — fall back, flag low confidence, never fail silently.”

Line 4 — if asked about regulation/compliance you do not know. “I have not worked under a regulated software process — FDA or IEC-style. I know the principle is traceability from requirement to test, and that is exactly the part of engineering I want to be trained in properly rather than guess at.”

20-minute clock plan

Clock	Phase	Your job
0:00–1:00	Greeting	Camera on, smile, “Good morning sir.” Confirm audio once, then let him lead.
1:00–2:30	Intro	§4.1, under 60 seconds. Plant LogiFlow, 73/68/78.3, Top 106, 470+ DSA . Then stop.
2:30–10:00	Project deep-dive	Architecture first, then whichever thread he pulls. Numbers only where you can defend them. Let him interrupt.
10:00–14:00	Technical probe	OOP / DBMS / OS, or a short coding or design ask. Narrate approach before syntax.
14:00–17:00	Fit / why GEHC	§4.2, 40–50 seconds. Bengaluru: unqualified yes.
17:00–19:00	Your questions	Two maximum from the interviewer brief.
19:00–20:00	Close	“Thank you sir, I enjoyed the conversation” — and mean it. Do not ask how you did.

If he runs short (10 minutes total). That is normal for a 20-minute grid with a schedule to keep. It is not a rejection signal. Make sure LogiFlow's architecture and one defensible number got said.

If he goes deeper than you can follow. Say: "I have not worked on that. The closest thing I have done is — can I tell you how I would approach it?" Then reason out loud. **Never bluff a cloud or systems answer with this interviewer.**

If he asks something you half-know. State the part you are sure of, mark the boundary explicitly, then reason forward. Half-knowledge stated honestly scores; half-knowledge stated confidently does not.

Night-before and morning checklist

Tonight (Thu 17 Sep)

- Say §4.1, §4.2, §4.3 out loud **three times each**, timed. Cut anything past 90 seconds.
- Write the **runtime polymorphism** snippet (§5.1) from memory on paper, once. Then Two Sum (§5.2).
- Re-read your own resume as an adversary: for every noun on it, can you answer one "why?" If not, prepare that one line tonight.
- Confirm your numbers: 73.0% accuracy, 68.0% delayed recall, 78.3% ROC-AUC (stratified holdout + 5-fold), +12.6% novelty, +20.9% serenity vs NSGA-3, Top 106, CGPA 9.05 (till 4th sem), LeetCode **470+**, contest max 1620, Web Wonders + Echelon finalist. **Never inflate.**
- Skim the interviewer brief once — then close it. Do not walk in trying to psychoanalyse him.
- Charge laptop and phone. Test Teams camera and mic tonight, not at 10:15.
- Lay out: resume printout, blank A4, pen, water. **Sleep by 23:30** — rested beats over-prepared.

Tomorrow morning (Fri 18 Sep, 10:20 slot)

- **09:30** — one calm read of §4 model answers. No new topics. Learning something new now only adds noise.
- **09:50** — eat something light. Do not interview hungry or over-caffeinated.
- **10:00** — close every app except Teams, your resume, and one browser tab. Silence phone, disable notifications.
- **10:10** — say the intro out loud once, at full volume, to warm your voice up.
- **10:17** — join the Teams link. Camera on, name set to **Bhatiya Kavya Vishnukumar**.
- **In the room** — sit up, look at the camera not the screen, speak slower than feels natural, and **stop talking when you have answered.**

The three ways interns lose this round — avoid all three.

1. **Inflating ownership.** "I built the whole thing" collapses under one follow-up. Name your components.
2. **Bluffing infrastructure.** He ran DevOps and hybrid cloud products for years. "I have not done that yet" is a scoring answer; a wrong confident answer is not.
3. **Talking past the answer.** In a 20-minute slot, every extra 30 seconds costs you a question you would have won.

You are ready. You have a globally recognised project (Top 106), honest metrics you can defend, SMSRF research, Docker on the resume, and 470+ DSA problems. Round I with an architect is about *how you think*. Think out loud, stay inside the resume, and stop when you have answered.

